

1 DIFF 算法的原理是什么？

回答重点

DIFF 算法的核心原理是通过比较新旧两棵虚拟 DOM 树，找出不同点，并且只更新必要的部分以达到高效更新真实 DOM 的目的。这种差异化更新策略能够显著提升性能，避免不必要的 DOM 操作。

扩展知识

1. 虚拟 DOM: 在 DIFF 算法中，虚拟 DOM (Virtual DOM) 是对真实 DOM 的一种抽象表示。虚拟 DOM 是 JavaScript 对象，它能够快速创建和更新，并且能够在内存中进行操作，避免频繁操作真实 DOM。

2. 核心步骤:

树的递归比较: 从根节点开始，递归地对比新旧虚拟 DOM 树，通过深度优先的方式逐层比较。

同层比较: 只对同一层级的节点进行比较，如果节点类型不同，则直接替换。

节点的复用和更新: 如果节点类型相同，则检查属性和子节点的变化情况，更新属性并递归地对子节点进行比较。

3. Vue 中的 DIFF 算法

- 在 Vue 中，虚拟 DOM 的 DIFF 比较主要发生在组件更新阶段。当数据发生变化后，Vue 会根据响应式数据模型产生新的虚拟 DOM 树，并通过 DIFF 算法与旧的虚拟 DOM 树进行对比找出变化部分，并以最小化的成本更新真实 DOM。

4. 优化策略:

- Key 的使用: 在 Vue 中，为了优化列表渲染的性能，通常会给每个列表项添加唯一的 key，这样能更高效地找到对应项并实现最小化更新。

批量更新: Vue 会收集多次数据变化，并且在一个事件循环中进行批量更新，这样可以避免重复的虚拟 DOM 比较和真实 DOM 更新。

5. React 与 Vue 的对比:

虽然 React 和 Vue 都使用虚拟 DOM 和 DIFF 算法，但实现细节有所不同。React 更侧重于函数式编程和不可变数据，Vue 则专注于响应式数据和模板语法。两者的 DIFF 算法核心思想相同，但在具体实现上可能会有所差异。

2 使用 Object.defineProperty 来进行数据劫持有什么缺点？

回答重点

使用 `Object.defineProperty` 来进行数据劫持(即数据绑定和响应式变更)虽然已经被广泛使用，但确实存在一些缺点，主要如下:

1) 无法监测数组变化: `Object.defineProperty` 对数组的一些变更操作，例如 `push`、`pop`、`shift`、`unshift`，无法检测到。这是一个比较大的限制，因为对于使用数组频繁的应用来说，这意味着某些变更不会被响应式绑定捕捉到。

2) 无法深层次劫持: 如果我们需要劫持一个嵌套结构(即对象中的对象)，我们需要递归地遍历和劫持所有的子属性，这会导致性能问题，尤其是在处理深层次嵌套对象时。

3) 不支持新属性的动态添加和删除: `Object.defineProperty` 不能监控到对象新增或者删除属性，Vue 为了解决这个问题，使用了诸如 `vue.set` 和 `vue.delete` 这样的包裹方法。

4) 性能开销大: 大量使用 `Object.defineProperty` 劫持数据时，会产生较大的性能开销，特别是在页面加载或者数据初始化阶段，需要频繁遍历对象和定义属性。

扩展知识

被这些缺点促使开发者去寻找更高效、更全面的解决方案。比如在 Vue3 中，`object.definePropertyProxy` 替代了，主要因为 Proxy 具有以下优点：

- 1) 全面的变化侦测能力: 能够侦测到数组的变动，以及对象属性的增加和删除，这解决了上述提 Proxy 到的 `object.defineProperty` 无法监控数组操作和新增删除属性的问题。
- 2) 更好的性能表现: 在处理深层对象嵌套时表现得更加优越，利用懒代理技术，可以有效减少 Proxy 能开销。
- 3) 语法更简洁: Proxy 是一种新的语法，更加现代化，代码可读性更强，而且避免了很多传统问题虽然 proxy 目前在现代浏览器中已经得到了很好的支持，但由于兼容性问题，它不能在某些老旧浏览器中工作。不过，通常应用在现代化项目开发中，结合 polyfil 也能较好解决这个问题。

3Vue Router 的核心实现原理是什么？

回答重点

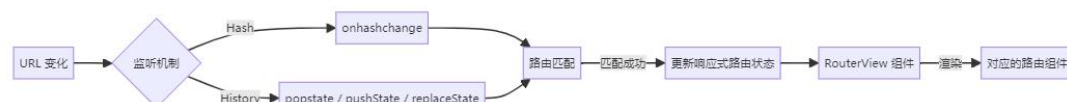
Vue Router 的核心原理主要围绕着以下几点：

1. 监听 URL 变化: Vue Router 会根据配置的模式 (Hash 模式或 History 模式) 监听浏览器地址栏 URL 的变化。

- Hash 模式: 利用 `onhashchange` 事件监听 URL 中 hash 部分的变化。

History 模式: 利用 HTML5 History API 中的 `pushState` 和 `replaceState` 方法改变 URL，并通过 `onpopstate` 事件监听浏览器的前进和后退操作。

2. 路由匹配: 当 URL 发生变化时，Vue Router 会遍历预先定义的路由配置表。它使用路径匹配算法 (如路径到正则表达式的转换) 来查找与当前 URL 最匹配的路由规则。
3. 状态更新: 一旦找到匹配的路由规则，Vue Router 会将当前路由的信息 (包括路径、参数、查询参数等) 更新到一个响应式对象中。这个对象通常可以通过 `useRoute()` 获取。
4. 视图渲染: `<router-view>` 组件会订阅这个响应式路由对象的变化。当路由信息更新时 `<router-view>` 会根据匹配到的路由规则，动态地渲染对应的组件到视图中，`<View>` 简单来说，就是监听 URL → 匹配路由规则 → 更新路由状态 → 渲染对应组件。



扩展知识

1) 两种模式的实现细节

Hash 模式

原理: URL 中的 #号(hash) 及之后的部分不会被发送到服务器。当 hash 改变时，会触发 `hashchange` 事件。Vue Router 监听这个事件，解析 hash 值，找到对应的路由规则并渲染组件。优点: 兼容性好，不需要服务器端额外配置。

缺点: URL 中带有 #，不太美观。

历史模式

原理: 利用 HTML5 历史应用程序接口的 `pushState` 和 `replace State` 方法来改变网址，这些方法不会触发页面刷新。通过监听弹出状态事件来响应浏览器的前进/后退操作。

优点: URL 更美观，符合常规 URL 形式。

缺点: 需要服务器端配置支持。当用户直接访问某个路由地址 (或刷新页面) 时，服务器

需要能够处理这个请求, 并将同一个 HTML 文件(通常是 index.html)返回给浏览器, 否则会报 404 错误。

2) 路由匹配机制

路由器内部维护一个路由记录(RouteRecord)列表。当 URL 变化时, 它会:

1. 解析当前 URL 路径。
2. 遍历路由记录列表。
3. 将每个路由记录的路径转换成正则表达式(或使用类似路径正则表达式的库)。
4. 使用转换后的正则表达式与当前 URL 路径进行匹配。
5. 如果匹配成功, 提取路径中的参数。
6. 考虑路由嵌套和优先级, 确定最终匹配的路由记录链。

3) 导航守卫

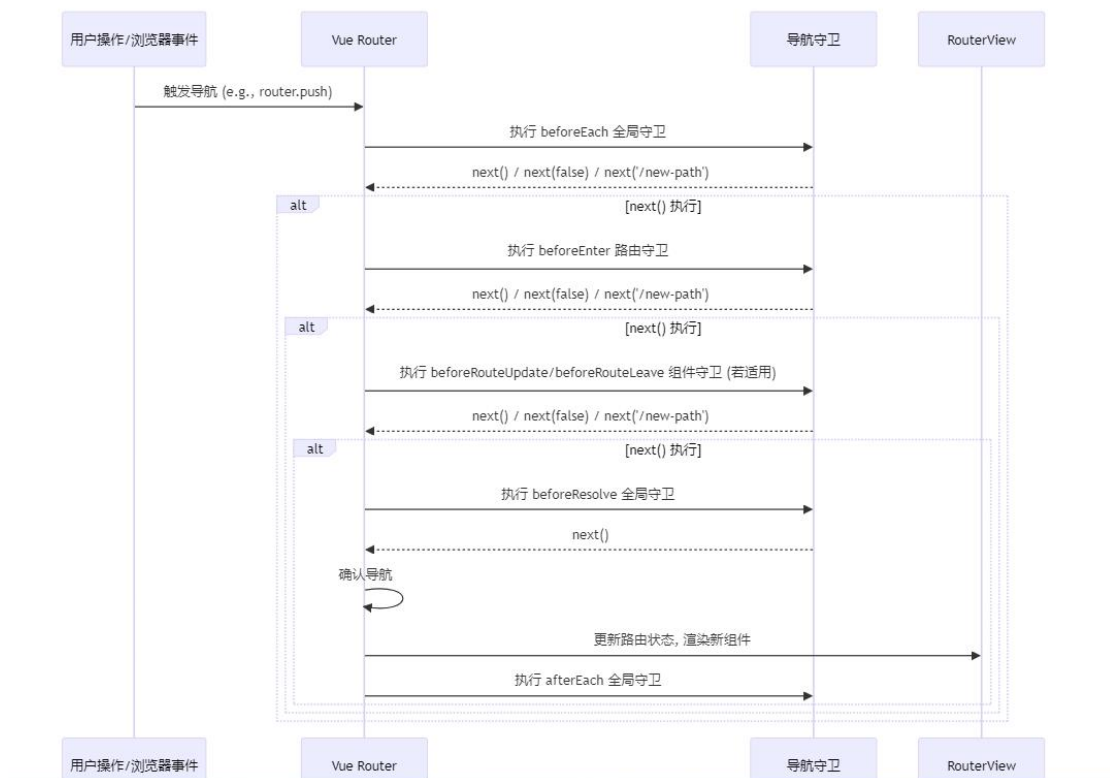
导航守卫允许我们在路由跳转的不同阶段执行自定义逻辑, 例如权限验证、数据获取等。

它们分为: 1. 全局守卫: `beforeEach`, `beforeResolve`, `afterEach`。作用于所有路由跳转。

2. 路由独享守卫: 在路由配置中直接定义 `beforeEnter`。

3. 组件内守卫: 在组件选项中定义 `beforeRouteEnter`, `beforeRouteUpdate`, `beforeRouteLeave`。

导航守卫的执行流程是一个异步队列, 可以通过 `next()` 函数来控制导航是继续、中断还是跳转到新的路由。



4) <router-link>和<router-view>

<router-link>: 用于声明式导航。它会被渲染成一个<a>标签(默认情况下), 并自动处理点击事件, 阻止默认的页面跳转, 调用 `router.push` 或 `router.replace` 来改变 URL 并触发导航。<router-view>: 路由出口组件。它根据当前匹配的路由信息, 动态渲染对应的组件。在嵌套路由中, 路由组件模板里的<router-view>会渲染子路由对应的组件

5) 相关文档

可以在 Vue Router 官方文档中找到更详细的信息!

路由匹配语法:

<https://router.vuejs.org/zh/guide/essentials/route-matching-syntax.html>

导航守卫:

<https://router.vuejs.org/zh/guide/advanced/navigation-guards.html>HTML5

History

模式:

<https://router.vuejs.org/zh/guide/essentials/history-mode.html>RouterView

组件 API:

<https://router.vuejs.org/zh/api/interfaces/RouterView.html>RouterLink

组件 API: <https://router.vuejs.org/zh/api/interfaces/RouterLink.html>

4Vue 的源码有哪些巧妙的设计?

回答重点

Vue 的源码设计中有很多巧妙之处,其中包括响应式系统、虚拟 DOM 以及模板编译。

下面分别简要介绍每个部分:

1) 响应式系统:Vue 的响应式系统基于 ES5 的 `object.defineProperty` 方法(Vue3 采用了 Proxy 更强大而且高效),可以在数据被访问和修改时进行劫持,从而做出相应的更新操作。这个设计使得

Vue 能够自动追踪依赖和更新视图,而不需要手动操作 DOM。2) 虚拟 DOM:Vue 使用虚拟 DOM 进行高效的 DOM 操作。虚拟 DOM 是对真实 DOM 的轻量级表示,通过对虚拟 DOM 进行比较(如 diff 算法),Vue 能够高效地确定需要更新的部分,并进行最小化的实际 DOM 操作,从而提升性能。

3) 模板编译:Vue 的模板编译步骤主要分为编译、优化和代码生成三个阶段。模板编译能够将模板字符串编译成渲染函数,从而在运行时只需要调用渲染函数生成虚拟 DOM。这种预编译的设计使得 Vue 的运行性能得以提升。

扩展知识

以上提到的只是 Vue 源码设计中一些比较突出的部分,实际上 Vue 作为一个成熟且流行的框架,还有很多细节和巧妙之处值得深入探究。下面是扩展的点:

1) 双向绑定:

Vue 的双向绑定在于它的 `v-model` 指令,这个指令的实现也依赖于响应式系统。`v-model` 可以说是封装了 input 元素的 `value` 属性以及 input 事件,从而实现数据和视图的同步更新。

2) 依赖收集:

Vue 的依赖收集机制很关键。它通过 `Dep` 和 `watcher` 实现,当一个响应式数据被读取的时候,它会把这个响应式数据所依赖的 `watcher` 收集起来。从而在这个数据更新时通知这些 `watcher` 进行更新。

3) 组件化设计:

Vue 的组件化是它的一大亮点。通过将 UI 拆分成独立的、可复用的组件,Vue 不仅提升了代码的可维护性,也使得开发者能够更高效地构建复杂的用户界面。每个组件都有自己的状态和生命周期,进一步增强了开发的灵活性。

4) 插件和生态系统:

Vue 有一个非常强大的插件机制,使得第三方库的集成变得非常容易。如 Vue Router、Vuex 这些官方库都是基于 Vue 的插件机制开发的,这样可以非常平滑地与 Vue 主框

架集成，同时因为统一了插件的开发模式，使得社区的开发者也可以方便地开发出各种各样的插件来扩展 Vue 的功能。

5) 性能优化：

Vue 在性能上也进行了深度优化。除了上面提到的虚拟 DOM 和模板编译，还有诸如静态节点树提升事件委托、懒加载等多种手段，确保 Vue 应用在大规模数据和复杂交互下依然能够保持高效的运行。

5 如何在 Vue3 中使用 defineAsyncComponent 实现异步组件加载？

回答重点

在 Vue 3 中使用 defineAsyncComponent 实现异步组件加载主要有以下几种方式：1) 基本用法：使用 ES 模块动态导入来异步加载组件

代码：import { defineAsyncComponent } from 'vue'

// 最简单的方式

```
const AsyncComp = defineAsyncComponent(() =>
  import('./components/MyComponent.vue')
)
```

2) 全局注册异步组件：

```
app.component('MyAsyncComponent', defineAsyncComponent(() =>
  import('./components/MyComponent.vue')
))
```

3) 局部注册异步组件：

// 在 Options API 中

```
import { defineAsyncComponent } from 'vue'

export default {
  components: {
    AdminPage: defineAsyncComponent(() =>
      import('./components/AdminPage.vue')
    )
  }
}
```

// 在 Composition API 的<script setup>中

```
import { defineAsyncComponent } from 'vue'
```

```
const AdminPage = defineAsyncComponent(() =>
  import('./components/AdminPage.vue')
)
```

4) 处理加载状态和错误状态:

```
const AsyncComp = defineAsyncComponent({
  loader: () => import('./components/AdminPage.vue'),
  loadingComponent: LoadingComponent,
  delay: 200,
  errorComponent: ErrorComponent,
  timeout: 3000
})
```